ENGINEERING BRIEF · AGENT G · NEW ONBOARDING

Roomio — Realistic Rendering

Make Roomio's lighting look photographic, not flat. The "ray tracing" goal — achieved the way real-time web 3D actually does it: physically-based materials, image-based lighting, soft shadows, ambient occlusion, glowing bulbs with bloom, and area lights. With an optional progressive path-traced "hero render" mode for beauty stills.

PBR + IBL + BLOOM | REAL-TIME, NOT TRUE RAYTRACE | OPTIONAL PATH-TRACE STILLS

Repo: `~/Desktop/personal/roomio` · **Your folder:** `/rendering`
Coordination: `/shared` + `roomio.txt` + `LEARNINGS.md` · **Agent D merges**
Date: June 29, 2026 · Works ON TOP of Agent E's lighting — coordinate tightly

Contents

1 Critical: What "Ray Tracing" Means Here 03

2 Onboarding & Relationship to Agent E 04

3 The Realism Stack (real-time) 05

4 Bulbs & Lights That Look Real 07

5 Optional: Progressive Path-Traced Stills 08

6 Performance & Gotchas 09

7 Coordination & Schema 10

8 Requirements & Acceptance 11

9 Milestones 12

1. Critical: What "Ray Tracing" Means Here

READ BEFORE ANYTHING ELSE

The goal was stated as "ray tracing for the lights." **True per-pixel ray tracing is not how real-time web 3D works** and isn't feasible live in a browser — even tracing a few thousand rays generally can't run in real-time. Three.js renders by rasterization. So you will NOT implement a real-time ray tracer. You will achieve the *look* of realistic, ray-traced lighting using the techniques that real-time WebGL actually uses: PBR + image-based lighting + soft shadows + ambient occlusion + bloom + area lights. That combination is what makes a scene read as photographic.

The one place real ray tracing fits: a **non-real-time "hero render"** mode that progressively path-traces a still image while the camera is held steady (covered in §5, optional). That's a beauty-shot feature layered on top — never the interactive default.

ONE-LINE SCOPE

Core: make the real-time scene photographic via PBR/IBL/shadows/AO/bloom/area-lights on top of Agent E's lighting. Optional: a progressive path-traced still for export-quality "hero" renders. No real-time ray tracer.

2. Onboarding & Relationship to Agent E

> You are **Agent G**, joining the Roomio build (A interiors, B detection/camera, C house+persistence, D coordinator, E lighting, F QA/dedup). First: read `roomio.txt` end-to-end + `LEARNINGS.md` fully, then post an onboarding entry. Code in `/rendering` ; commit only your own paths; **Agent D merges**.

YOU SIT ON TOP OF AGENT E — DON'T COLLIDE

Agent E owns the lights themselves (default room lights, the sun, the time-of-day bar, shadow setup). You own how those lights *look*: the materials they fall on, the environment lighting, tone mapping, post-processing (bloom/AO), and the renderer's realism settings. There is a real seam here. Coordinate via `/shared` and `roomio.txt` — agree who sets what (e.g. E places a bulb; you make it glow and give the room PBR materials + IBL). Don't both set renderer shadow config; settle ownership early via a REQUEST -> AGENT-E.

3. The Realism Stack (real-time)

Realism is a layered recipe. Apply in this order; each layer compounds.

Layer	Technique	What it buys
PBR materials	MeshStandardMaterial / MeshPhysicalMaterial (roughness + metalness workflow)	Surfaces respond to light like real materials. Physical adds clearcoat, sheen (velvet), transmission (glass), IOR. The single biggest realism lever.
Image-based lighting (IBL)	HDR environment map → `scene.environment` via PMREMGenerator	Soft ambient bounce + real reflections for all PBR materials at once. Efficient and the key to "it looks lit by a real room."
Tone mapping	ACESFilmicToneMapping (most popular cinematic look); sRGB output	Squashes HDR range into a filmic, photographic curve. Without it, bright areas clip and look flat/CG.
Soft shadows	PCFSoftShadowMap (coordinate with E who owns the sun)	Antialiased contact shadows that ground objects.
Ambient occlusion	SSAO/GTAO post-pass, and/or baked aoMap on assets	Soft contact darkening in crevices/corners — huge perceived-realism gain for little cost.
Bloom	UnrealBloomPass on bright emissive areas	Makes bulbs/windows glow and bleed light like real bright sources (see §4).
Area lights	RectAreaLight (+ the required uniforms lib)	Soft, realistic falloff from panel/window-shaped sources — far more natural than point lights for interiors.

```
// IBL + filmic tone mapping — the realism foundation
renderer.toneMapping = THREE.ACESFilmicToneMapping;
renderer.outputColorSpace = THREE.SRGBColorSpace;
new RGBELoader().load('studio.hdr', (hdr) => {
  const env = new THREE.PMREMGenerator(renderer).fromEquirectangular(hdr).texture;
  scene.environment = env;          // all PBR materials now lit by it
});
const mat = new THREE.MeshStandardMaterial({ roughness: 0.4, metalness: 0.0, envMapIntensity: 1.0 });
```

4. Bulbs & Lights That Look Real

The brief's headline ask: bulbs and lights that look realistic. In real-time that's a combination of an *emissive* fixture, a *light* source, *bloom*, and physically-correct *falloff*.

- **Emissive fixture mesh:** the visible bulb/lamp gets a material with an `emissive` color/intensity so it appears to glow on its own (independent of being lit).
- **Bloom post-pass:** run UnrealBloomPass so emissive bulbs and bright windows bleed a soft halo — this is what sells "that bulb is actually on." Tune threshold so only genuinely bright things bloom.
- **Physically-correct falloff:** use the renderer's physical light units so intensity falls off with distance like real light. (Modern Three.js r155+ defaults to physical units — tune to it, don't copy old intensities.)
- **Right light type per fixture:** a bare bulb → PointLight; a directional lamp/spot → SpotLight with penumbra for soft cone edges; a panel/window → RectAreaLight for soft area falloff. Coordinate with E on which lights E already places vs which you enhance.
- **Warm emissive + warm light:** match the emissive color to the light color (warm bulbs glow warm). Ties into E's warm/cool control.

COORDINATE THE SEAM WITH E PRECISELY

E creates lights; you make them *read* as real (emissive fixtures + bloom + falloff + materials they illuminate). Agree the contract: E's light objects expose enough (position, color, intensity, type) that you can attach the matching emissive fixture + bloom treatment. Don't duplicate E's lights — enhance them.

5. Optional: Progressive Path-Traced Stills

The one place *real* ray tracing belongs — a non-interactive "hero render." When the user holds the camera still and clicks "Render beauty shot," a WebGL path tracer progressively refines a truly ray-traced image (real global illumination, soft shadows, accurate reflections) over a few seconds, then offers it as a high-quality export.

- Use an existing WebGL path-tracing library (e.g. `three-gpu-pathtracer`) rather than writing a tracer. It runs on the same scene/PBR materials.
- **Strictly non-real-time:** only runs while the camera is static; pauses/falls back to the rasterized view the moment the user moves. Never the interactive default.
- Pairs naturally with Agent C's image-export / showcase features — the path-traced still is the premium export.
- Scope this as a clearly-separate stretch phase after the real-time stack is solid. Don't start here.

WHY THIS SPLIT IS CORRECT

Real-time realism (PBR/IBL/bloom) makes the whole app look great while staying interactive. Path tracing gives one gorgeous still on demand. Trying to path-trace the live, editable scene would tank framerate and fight E's real-time lighting — keep them cleanly separated.

6. Performance & Gotchas

Gotcha	Handling
Color space wrong	Color textures must be `SRGBColorSpace`; data maps (normal/roughness/AO) stay linear. Wrong color space = washed-out or muddy materials.
No tone mapping	Without ACESFilmic + sRGB output, HDR clips and the scene looks flat/CG. Always set both.
Too many real-time lights	Each shadowed light is costly. Lean on IBL for ambient + the sun (E) for the key shadow; keep extra shadow-casters few.
RectAreaLight setup	Needs `RectAreaLightUniformsLib.init()` and only works with Standard/Physical materials; no shadows from it.
Bloom over-blooming	Tune the bloom threshold so only truly bright emissive/HDR areas glow, or everything turns hazy.
aoMap needs uv2	Ambient-occlusion maps require a second UV set on the geometry — coordinate with the texture/asset agent (H) and F.
Mobile cost	Post-processing (SSAO/bloom) is GPU-heavy; offer a quality toggle (high/medium/low) so low-end devices stay smooth.

RECORD WORKING VALUES

Tone-mapping exposure, bloom threshold/strength, envMapIntensity, SSAO radius — all scene-scale dependent. Tune against a real furnished room and record the values in `LEARNINGS.md` so E, H and future you don't re-derive them.

7. Coordination & Schema

Stack

Need	Tool	Note
PBR materials	MeshStandardMaterial / MeshPhysicalMaterial	Physical for glass/velvet/clearcoat.
IBL	RGBELoader + PMREMGenerator	HDR env → scene.environment.
Post FX	EffectComposer: UnrealBloomPass, SSAO/GTAO, output pass	Or postprocessing lib / R3F drei if app is R3F (confirm via LEARNINGS.md).
Area lights	RectAreaLight + RectAreaLightUniformsLib	Coordinate with E.
Path tracer (optional)	three-gpu-pathtracer	Hero stills only.

Schema (publish to /shared)

```
{
  "render_quality": "high|medium|low",
  "tone_mapping": "ACESFilmic", "exposure": 1.0,
  "ibl": { "env": "studio.hdr", "intensity": 1.0 },
  "post": { "bloom": {"threshold":0.9,"strength":0.6}, "ssao": true },
  "hero_render": { "enabled": false, "samples": 256 }
}
```

Coordination

- Code in `/rendering`; only other writable places `/shared`, `roomio.txt`, `LEARNINGS.md`. Commit own paths; **D merges**.
- Read `roomio.txt` + `LEARNINGS.md` each cycle. Watch `[USER -> AGENT-G]` (top priority) and `ISSUE -> AGENT-G` from D.
- Post `REQUEST -> AGENT-E` (settle the lights-vs-look seam; renderer/shadow ownership), `REQUEST -> AGENT-H` (assets need PBR map slots + uv2 for AO), `REQUEST -> AGENT-A` (apply PBR materials to archetypes), `REQUEST -> AGENT-B` (path-traced still feeds the camera/export).

8. Requirements & Acceptance

ID	Requirement	Area
R-1	Scene uses PBR materials (Standard/Physical) with correct color space.	PBR
R-2	IBL via an HDR env map drives ambient + reflections for all PBR materials.	IBL
R-3	ACESFilmic tone mapping + sRGB output enabled.	Tone
R-4	Ambient occlusion (SSAO/GTAO or baked aoMap) for contact darkening.	AO
R-5	Bulbs glow (emissive) and bloom; bloom threshold tuned to avoid haze.	Bulbs
R-6	Physically-correct light falloff; right light type per fixture; area lights for panels/ windows.	Bulbs
R-7	Enhances Agent E's lights without duplicating them; renderer ownership settled with E.	Seam
R-8	Quality toggle (high/medium/low) keeps low-end devices smooth.	Perf
R-9	Optional: progressive path-traced hero still while camera is static; falls back on move.	Stretch
R-10	Code in /rendering; coordinate via /shared+roomio.txt+LEARNINGS.md; D merges; no edits to others' source.	Coord

Acceptance test

A furnished room renders with PBR materials lit by an HDR environment, filmic tone mapping, soft contact shadows/AO, and bulbs that visibly glow with a soft bloom halo — clearly more photographic than the flat baseline, side by side. Turning a bulb on/off (E's control) changes both the light and its glow. Switching quality to "low" drops post-FX and holds framerate on a weaker device. Optionally, holding the camera still and clicking "Render beauty shot" progressively produces a path-traced still that's exportable; moving the camera returns to the real-time view. No real-time ray tracer; renderer ownership with E is clean.

9. Milestones

Milestone	Deliverable	Definition of done
G0	Read roomio.txt + LEARNINGS.md; scaffold /rendering; settle renderer/light seam with E; publish render schema; onboarding entry	Seam agreed with E; schema in /shared.
G1	PBR materials + IBL env map + ACESFilmic tone mapping + sRGB	Room visibly lit by environment, filmic look.
G2	Ambient occlusion + soft shadow integration (with E)	Contact darkening + grounded objects.
G3	Emissive bulbs + bloom + correct falloff + area lights	Bulbs glow realistically; values logged to LEARNINGS.md.
G4	Quality toggle + mobile perf pass	high/medium/low; low-end stays smooth.
G5	Optional: progressive path-traced hero render + export hook (with B/C)	Static-camera beauty still, exportable, falls back on move.

One-line summary. In `/rendering`, make Roomio photographic the way real-time WebGL actually does it: PBR materials + HDR image-based lighting + ACESFilmic tone mapping + ambient occlusion + emissive bulbs with bloom + area lights, layered on Agent E's lighting (settle the seam first). Optional progressive path-traced "hero render" for export stills. No real-time ray tracer; coordinate via /shared + roomio.txt + LEARNINGS.md; D merges.

End of brief · Roomio Realistic Rendering (Agent G) · Real-time realism, not a real-time ray tracer; tune and record values in LEARNINGS.md.